LR-Agent / Agent 后端入门

第05章
FastAPI流式接口与请求续跑

从 HTTP 输入到 SSE 输出，理解工具挂起、结果配对与请求重建

读者起点与本章边界

前置：已完成第 1、2 章基础实验。只围绕后端输入、决策、执行与验证学习；遇到新名词先看定义和小例子。真实截图、源码阅读图与概念示意分别注明。

学完本章应该能够

- 读懂请求模型、依赖注入与流式响应
- 解释 tool_pending 和 tool_call_id 配对
- 用离线实验验证消息去重与确认口径
- 准确区分请求续跑、协作取消与图持久恢复

源码核对：2026-10-17，工作区基线 d5dd009；本轮包含已验证的日志断管道修复。正式环境 LangChain 1.4.2 / LangGraph 1.2.12。

本章目录与实验准备

5.1 从一次请求开始：后端究竟接了什么 . . . 3
5.2 Pydantic：让输入先过一道格式检查 . . . 4
5.3 Depends：由框架把依赖交给函数 . . . 5
5.4 SSE：一个响应里按帧交付结果 . . . 6
5.5 async、await、yield 分别在做什么 . . . 7
5.6 完整实验一：校验一个真正的流式路由 . . . 8
5.7 一张事件表读懂输出，不只盯着文字 . . . 9
5.8 tool_pending：暂停的是业务交接 . . . 10
5.9 tool_call_id：把调用与结果配成一对 . . . 11
5.10 完整实验二：重复投递后消息会增加吗 . . . 12
5.11 幂等要拆成“消息层”和“执行层” . . . 13
5.12 完整实验三：待确认不是已经写盘 . . . 14
5.13 取消：尽快停止，并负责收尾 . . . 15
5.14 源码导航：只沿一条请求路径阅读 . . . 16
5.15 练习：给协议故障一个确定答案 . . . 17
5.16 面试表达：流式与续跑不是两个名词 . . . 18
5.17 源码图解：从函数签名读到返回值 . . . 19
5.18 真实接口图：把后端符号与请求对上 . . . 20

每个实验包含全部导入、输入与执行。固定模型和教学数据只验证机制；源码节选只用于阅读。请先预测输出，再运行并改变一个输入。

```
Set-Location D:\Study\LR-Agent
$env:PYTHONPATH = Join-Path (Get-Location) 'LR-Agent\vendor\local-agent'
$env:LANGSMITH_TRACING = 'false'
$env:LANGCHAIN_TRACING_V2 = 'false'
$agentPy = 'D:\Study\LR-Agent\.venvs\langchain-main\Scripts\python.exe'
```

5.1 从一次请求开始：后端究竟接了什么

场景：你输入“给 demo.jpg 标注目标”，后端不能只接一段字符串。它还需要知道本轮模型设置、已有消息、工作区快照、任务 ID，以及是否携带上一轮工具结果。第 3、4 章研究的是 Agent 内部循环，本章研究循环如何跨越 HTTP 边界。

HTTP 是请求与响应的通信协议；API 是约定的服务入口；JSON 是字段与值的交换格式。调用方发 POST 请求，FastAPI 根据路径找到函数，把请求体校验为 Python 对象，再返回流。调用方只作为协议参与者，不需要学习其界面实现。

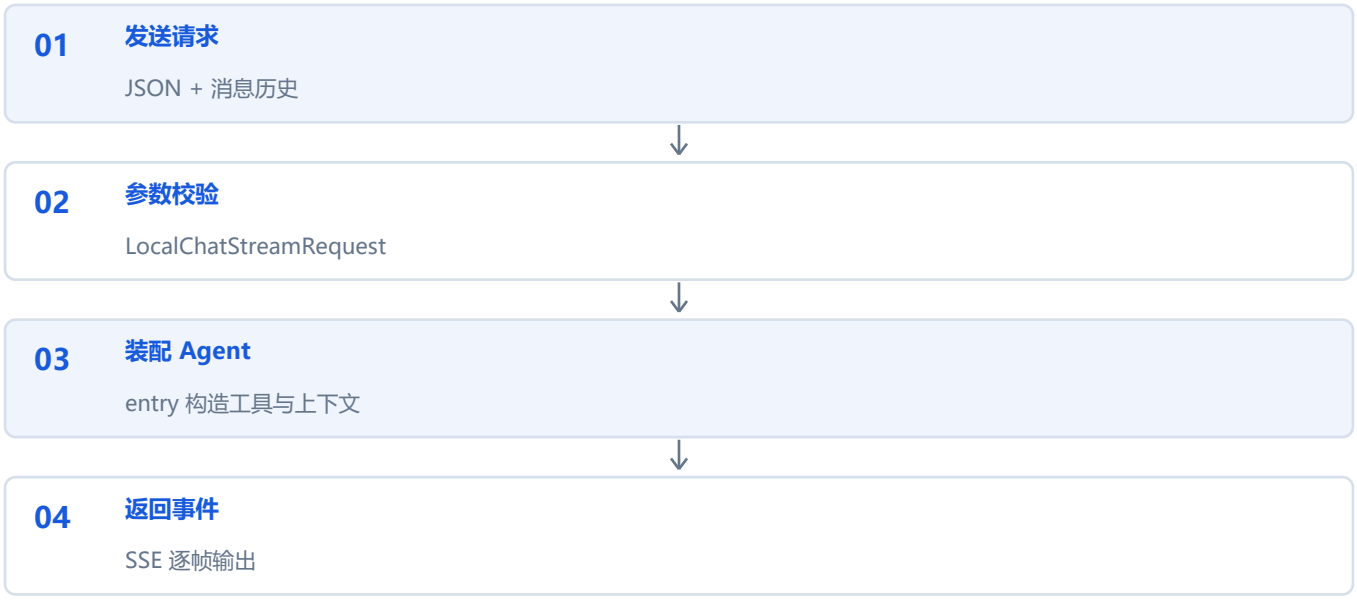


图5-1 实际后端调用链的简化视图

名词	白话解释	项目对应
请求体	这次输入的一包数据	LocalChatStreamRequest
路由	URL 对应的处理函数	local_chat_stream
响应流	结果陆续返回	StreamingResponse

本章验收

能画出请求到 SSE 的路径；解释一次工具挂起与续跑；运行三个离线实验；准确说明没有服务端图持久恢复。

5.2 Pydantic：让输入先过一道格式检查

Pydantic 依据类里的类型标注与 Field 规则检查数据。BaseModel 是数据模型的父类；继承它以后可以通过 model_validate 读取字典，或通过构造函数传入字段。Field(min_length=1) 表示字符串至少有一个字符。校验解决“数据是否符合约定”，不会证明 URL 可访问、模型可调用或任务已获授权。

字段	输入含义	阅读重点
messages	历史消息列表	role 区分消息角色
client_job_id	本次运行的标识	用于取消，不是工具调用 ID
client_tool_results	执行方回传列表	每项有独立 tool_call_id
client_context	工作区及业务状态	可为空，决定工具路径

源码节选：LR-Agent/vendor/local-agent/app/schemas/agent.py::ClientToolResult

```python
class ClientToolResult(BaseModel):
    """客户端执行工具后返回的结果。"""
    tool_call_id: str = Field(min_length=1, max_length=128)
    name: str = Field(min_length=1, max_length=128)
    result: str = Field(
        description="工具执行结果（JSON 序列化字符串）"
    )
```

str 是字符串；list[ClientToolResult] 是元素需符合该模型的列表；X | None 表示允许值为None；是否可省略取决于默认值，例如 = None。result 当前是 JSON 字符串，不是任意字典，读取里面的状态还要 json.loads。此节源码节选省略相邻类与导入，不能直接当作完整脚本运行。

5.3 Depends：由框架把依赖交给函数

依赖注入并不神秘：处理请求时，某个函数需要配置，框架先调用配置提供函数，再把返回值传进去。它减少每个路由手工读取配置的重复。项目的 SettingsDep 是“Settings 类型 + Depends(get_settings)”的别名。settings 是参数名，真正的值由 FastAPI 解析依赖得到。

源码节选：LR-Agent/vendor/local-agent/app/core/deps.py::SettingsDep; app/api/v1/agent.py::local_chat_stream

```python
SettingsDep = Annotated[Settings, Depends(get_settings)]

async def local_chat_stream(
    body: LocalChatStreamRequest,
    settings: SettingsDep,
) -> StreamingResponse:
```

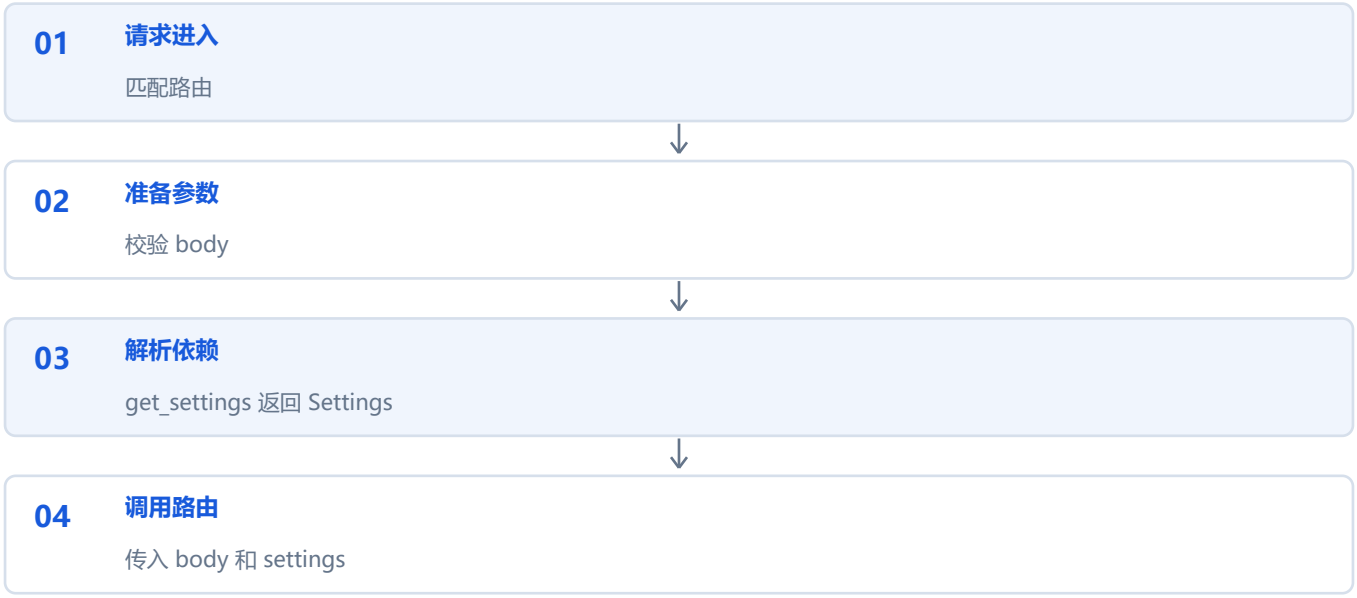


图5-2 概念示意：依赖注入的先后关系

Annotated 可以理解为“给一个类型附加框架可识别的说明”；Depends(...) 告诉 FastAPI 去调用谁。返回标注 -> StreamingResponse 描述预期返回类型，本身不会自动创建响应。项目本机服务另有访问令牌校验；配置注入和访问鉴权是两件事。

初学者常见误会

不要在普通脚本里直接调用 local_chat_stream(body) 并期待 Depends 自动生效。依赖解析发生在框架处理请求时；手工调用函数需要显式传入参数。

5.4 SSE：一个响应里按帧交付结果

SSE 即 Server-Sent Events，服务器发送事件。这里一次 POST 打开响应流，服务器不断写出事件文本。项目以 data: 开头，后面是 JSON，再用两个换行结束一帧。JSON 内的 type 才是项目业务事件名。它不是每生成一个 token 就重新发一次 HTTP 请求。

层次	示例	作用
HTTP 头	text/event-stream	声明流内容类型
SSE 文本	data: {...} + 空行	定义帧边界
业务 JSON	type=text_delta	描述这帧含义

源码节选：LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::sse_frame（仅重排表达式换行）

```python
def sse_frame(payload: StreamEventPayload) -> str:
    return (
        "data: "
        + json.dumps(payload.to_sse_dict(), ensure_ascii=False)
        + "\n\n"
    )
```

to_sse_dict 把 Python 模型变为协议字典，并按项目约定处理字段名；ensure_ascii=False 保留可读中文。拼接出的 str 由 StreamingResponse 写入响应。不能假设网络收到的一个数据块恰好等于一帧，接收方应按空行累计解析。

done 的边界

done 表示本次响应流收尾，不保证整个标注任务都已保存。工具挂起、用户待确认和取消后的收尾，都需结合业务事件和状态解释。

5.5 async、await、yield 分别在做什么

async def 声明异步函数；调用普通异步函数先得到协程对象，await 才等待它执行完成。yield 在函数中交出一个值并暂停当前位置；async def 中使用 yield，就得到可异步迭代的生成器，用 async for 逐项读取。它们不是“自动开线程”的开关。

写法	现在发生什么	返回或消费对象
await fn()	等待一次异步结果	一个最终值
yield frame	交出一项后可再继续	生成器的一项
async for x in stream	逐个等到流事件	消费异步迭代器
finally	成功或失败都收尾	关闭流、释放资源



图5-3 概念示意：流式响应的拉取关系

项目 frames 是事件源；async for frame in frames 逐帧处理；yield sse_frame(frame) 转成 HTTP 文本。取消时只是 break 并不足以保证上游停止，因此 finally 中还显式 aclose。同步的耗时 I/O 如果直接写在异步函数里，仍可能阻塞事件循环。

先理解责任，再记语法

await 对应“等结果”，yield 对应“交一项”，async for 对应“逐项消费”。本章不要求实现协程调度器，但要能解释流为什么还需要关闭。

5.6 完整实验一：校验一个真正的流式路由

完整教学实验 ch05_sse。在 study 自行新建 my_ch05_sse.py，复制全部代码。

```python
import json
from fastapi import FastAPI
from fastapi.testclient import TestClient
from fastapi.responses import StreamingResponse
from pydantic import BaseModel, Field
from app.agent.langchain_runtime.entry import sse_frame
from app.schemas.agent import StreamEventPayload
class Input(BaseModel):
    text: str = Field(min_length=1)
app = FastAPI()
@app.post('/learn')
async def learn(body: Input):
    async def events():
        yield sse_frame(StreamEventPayload(
            type='text_delta', content=body.text))
        yield sse_frame(StreamEventPayload(type='done'))
    return StreamingResponse(events(),
                             media_type='text/event-stream')
web = TestClient(app)
assert web.post('/learn', json={'text': ''}).status_code == 422
response = web.post('/learn', json={'text': 'hello'})
frames = [json.loads(x[6:])
          for x in response.text.strip().split('\n\n')]
assert [x['type'] for x in frames] == ['text_delta', 'done']
print('HTTP', response.status_code, '| invalid=422')
print('SSE:', [x['type'] for x in frames])
```

运行：& $agentPy ".\study\my_ch05_sse.py"（工作区根目录）

本轮实际输出：

```
HTTP 200 | invalid=422
SSE: ['text_delta', 'done']
```

先解释新语法：@app.post 将函数登记为 POST 路由；嵌套的 events 函数暂不执行。yield 每次交出一帧并保留当前位置，StreamingResponse 负责消费生成器。

输入 text 为空时由 Pydantic 校验为 422；合法输入被编码成 text_delta，随后是 done。sse_frame 是项目真实编码器，示例路由是教学路由。

TestClient 在进程内调用 ASGI 应用，不监听端口、不访问模型；它能验证响应结构，不能证明首字延迟、代理缓冲或真实网络取消。

5.7 一张事件表读懂输出，不只盯着文字

标注助手的响应不只有最终回答。准备工具、模型文字、工具开始、执行结果与挂起请求都可能出现在同一条流里。事件协议负责表达“发生了什么”；调用方如何显示它不属于本章学习范围。相同 type 的事件可能出现多次，不能把第一条 text_delta 当作完整答案。

事件	后端表达的事实	不能据此推断
preparing	正在准备某个阶段	工具已执行成功
text_delta	新增一段回答文字	文件已经落盘
tool_start / tool_result	调用及其返回内容	检测精度一定达标
tool_pending	需交给外部执行方	已执行并保存
done	这次流已经收尾	整个业务一定完成



图5-4 实际职责简化：内部事件到外部协议

sse_adapter.py 将框架 messages、updates、custom 映射到业务事件；entry.py 再编码 SSE 并发出 done。这样框架升级时，协议适配层可以承担变化，外部参与者不必直接理解 LangGraph 的内部事件形状。

5.8 tool_pending：暂停的是业务交接

某些工具由外部执行方掌握实际执行环境。后端模型可以提出 auto_annotate 调用，但当前 ASYNC 类工具不在后端实际执行。业务中间件记录挂起计划，流适配器发 tool_pending。ASYNC 是项目的执行归属分类，不等同 Python 的 async def。



图5-5 实际跨请求续跑协议的简化时序

tool_pending 中的参数必须来自已解析和合批后的计划，不能随意从未完成的模型 token 中截取。客户端工具调用有 tool_call_id、name 和 arguments。执行方回传同一 ID，后端才能把结果对上原调用。

图上没有隐藏的暂停线程

请求 1 可以结束；请求 2 重新携带所需历史和结果。当前没有把某个图运行现场保存进 checkpointer，也没有用 Command(resume=...) 唤醒它。

5.9 tool_call_id：把调用与结果配成一对

假设同一个工具要处理 a.jpg 和 b.jpg，两次 name 都是 auto_annotate。若只按名字回填结果，会分不清哪张图片已经处理。tool_call_id 是一次调用的唯一标识；ToolMessage 使用这个标识回答对应 AIMessage.tool_calls。client_job_id 则用于定位本次运行的取消事件，二者粒度不同。

标识或字段	描述什么	示例
client_job_id	本次请求运行	job-demo-1
tool_call_id	某一次工具调用	call-1
name	执行哪类工具	auto_annotate
message_id	某条业务历史消息	m-demo-1



图5-6 实际消息重建：先匹配，再补齐

历史缺少对应声明时，现有兼容函数会补合成 AIMessage 与 ToolMessage 对。它解决消息格式衔接，不证明任意回传都可信。课程实验使用固定演示 ID，不打印真实请求密钥、访问令牌或账号数据。

5.10 完整实验二：重复投递后消息会增加吗

完整教学实验 ch05_resume。在 study 自行新建 my_ch05_resume.py，复制全部代码。

```python
import json
from langchain_core.messages import AIMessage, ToolMessage
from app.schemas.agent import ClientToolResult
from app.agent.langchain_runtime.resume import (
    prepare_resume_turn,
)
declared = AIMessage(content='', tool_calls=[{
    'id': 'call-1', 'name': 'auto_annotate',
    'args': {'paths': ['demo.jpg']},
}])
result = ClientToolResult(
    tool_call_id='call-1', name='auto_annotate',
    result=json.dumps({'proposal_pending': True}),
)
first = prepare_resume_turn([declared], [result])
again = prepare_resume_turn(first.messages, [result])
tools = [x for x in again.messages
         if isinstance(x, ToolMessage)]
assert len(first.messages) == len(again.messages) == 2
assert len(tools) == 1 and tools[0].tool_call_id == 'call-1'
assert 'call-1' in again.context.completed_tool_ids
print('messages:', len(first.messages), len(again.messages))
print('tool_results:', len(tools))
print('completed:', sorted(again.context.completed_tool_ids))
```

运行：& $agentPy ".\study\my_ch05_resume.py"（工作区根目录）

本轮实际输出：

```
messages: 2 2
tool_results: 1
completed: ['call-1']
```

AIMessage 中的 tool_calls 声明“希望执行哪个调用”；ClientToolResult 携带执行方回传。二者依靠 call-1 对齐，不依靠工具名对齐，因为同一工具可以调用多次。

prepare_resume_turn 返回 ResumeTurn 对象，其中 messages 补 ToolMessage，context.completed_tool_ids 供执行门控使用。第二次输入包含第一次的消息历史，因已有结果而跳过追加。

本实验验证历史中已有结果时的重复投递过滤及已完成 ID 播种。它没有运行模型、标注或写盘，也没有验证同一批内重复 ID、新 ID 重试和多进程一致性。

5.11 幂等要拆成“消息层”和“执行层”

幂等的直觉是“同一件事重试后，业务结果不应多做一遍”。但不同层的问题不同：消息层要避免重复加入 ToolMessage；执行层要避免模型复读旧调用后再次执行。项目 prepare_resume_messages 过滤已有结果 ID，prepare_resume_turn 又把完成 ID 写进 AgentContext，供中间件 already_completed 分支使用。

层次	保护手段	本章证据
消息重建	历史已有结果则跳过	实验二重复回传
执行抑制	completed_tool_ids 门控	源码确认，未跑真实工具
外部副作用	执行方需要自身约束	本章不证明全局 exactly-once
业务状态	确认后再按已写盘描述	实验三验证消息标志

不要把这个机制描述成“任意重试都全局恰好执行一次”。历史缺失、同一批内重复 ID、给旧任务生成新 ID、多个服务进程和外部执行成功但回包丢失，都需要分别验证。本章实验只覆盖“回传 ID 已经出现在输入历史结果里”的路径。

面试追问

为什么去重不能只用工具名？因为同名工具可以合法处理不同输入。为什么不能只在提示词写“不要重复”？因为模型仍可能重复提出调用，必须有执行前的确定性检查。

5.12 完整实验三：待确认不是已经写盘

完整教学实验 ch05_result_state。在 study 自行新建 my_ch05_result_state.py，复制全部代码。

```python
import json
from app.agent.context_service import (
    enrich_resume_tool_result,
)
raw = json.dumps({
    'proposal_pending': True, 'file_written': False,
})
pending = json.loads(enrich_resume_tool_result(raw))
applied = json.loads(enrich_resume_tool_result(
    raw, proposals_applied=True,
))
skipped = json.loads(enrich_resume_tool_result(
    '{"status":"skipped","proposal_pending":false}',
    proposals_applied=True,
))
assert pending['proposal_pending'] is True
assert pending['file_written'] is False
assert applied['proposal_pending'] is False
assert applied['file_written'] is True
assert 'file_written' not in skipped
print('pending:', pending['proposal_pending'], pending['file_written'])
print('applied:', applied['proposal_pending'], applied['file_written'])
print('skipped_has_written:', 'file_written' in skipped)
```

运行：& $agentPy ".\study\my_ch05_result_state.py"（工作区根目录）

本轮实际输出：

```
pending: True False
applied: False True
skipped_has_written: False
```

json.dumps 将 Python 字典转为字符串；json.loads 将结果字符串还原为字典。is True 检查布尔值，不能把字符串 "false" 当作 False。

proposals_applied 是仅限关键字传参的开关，默认 False。生产调用方从本请求 TaskPhaseContext 推导它；示例直接传 True 只是演示分支。

只有已确认且原结果确实 proposal_pending=true，才修正为已写盘口径。skipped 没有提案，不应被包装成写盘成功。该函数修改消息内容，不执行文件写入。

5.13 取消：尽快停止，并负责收尾

取消通常是协作式的：某处记录“请停止”，运行中的代码在检查点看到后退出。项目 /chat/cancel 按 client_job_id 找到进程内 asyncio.Event 并 set；流生成器调用 is_cancelled 检查标志。asyncio.Event 是协程间共享信号；set 不会把所有外部 HTTP 请求或已完成写入自动倒回。



图5-7 实际取消流程的简化视图

动作	解决的问题	边界
Event.set	通知本进程任务	不是跨进程取消总线
break	停止当前消费循环	不必然自动关上游
aclose	关闭异步生成器	不撤销已生效副作用
finally 清理	移除本轮注册项	不等于持久审计记录

cancel 接口返回 ok=true，表示请求已按接口约定处理；如果 ID 不存在也会返回这个值，所以不能当成“任务确实取消成功”的强证明。用户断开连接、模型 SDK 停止、长工具执行结束和业务回滚是不同问题，后续若要承诺可靠取消，应为每种状态补观测和失败测试。

5.14 源码导航：只沿一条请求路径阅读

打开真实文件时不要从第一行通读上千行。先确认路由参数和 media_type，再跳到 entry 的生成器，最后看 resume 的两个输入输出。下面是实际装配关系的节选；省略了工具和视觉能力装配，不能据此判断完整业务实现。

源码节选：LR-Agent/vendor/local-agent/app/api/v1/agent.py::local_chat_stream

```
return StreamingResponse(
    stream_candidate_engine_sse(
        body,
        settings,
        is_cancelled=None,
    ),
    media_type="text/event-stream",
)
```

文件 / 符号	输入 → 输出	本次阅读问题
agent.py::local_chat_stream	请求对象 → HTTP 流	由谁选择引擎?
entry.py::stream_candidate_engine_sse	请求 → SSE 字符串	done、取消由谁收尾?
resume.py::prepare_resume_turn	历史+结果 → ResumeTurn	消息和完成集怎样重建?
sse_adapter.py::stream_agent_events	框架事件 → 业务事件	pending 与 result 怎样区分?

返回值 ResumeTurn 包含 messages、context 和 completed_tool_ids；它不是数据库记录，也不是历史线程句柄。源码里的 candidate 是保留的引擎配置名，目前是正式默认路径；不要把名称误读成未启用实验。建议真实截图：local_chat_stream 的函数签名与 StreamingResponse 返回段，仅显示源码，不显示请求配置。

5.15 练习：给协议故障一个确定答案

练习输入	先作判断	参考思路
同名工具连续两次调用	是否合并两个结果?	按 ID 区分，名字不能代替 ID
已有 ToolMessage 再回传同 ID	是否再补一条?	输入历史过滤，应保持数量
pending=true 但未确认	能说已保存吗?	不能，检查状态再组织回答
取消接口返回 ok=true	能说已回滚吗?	不能，取消与回滚独立

动手一：修改实验二的第二次输入为 [declared]，观察因为缺少已有 ToolMessage，函数是否仍需要补结果。参考：它按本次请求重建，不会记住上一次函数调用；输入历史责任很重要。动手二：把 call-1 改成 call-2，观察工具结果与原声明是否仍匹配，解释为何兼容函数会补合成声明。

再做一个异常分析：请求已发 tool_pending，执行方成功但回传丢失。你会优先补“稳定调用 ID、执行侧结果记录与重放查询”，还是直接让模型重做？参考：先恢复可观测的执行事实，再决定重试；模型没有外部副作用的可靠记忆。此处是改造讨论，教材没有实现这些能力。

可留下的个人证据

保存自己运行的命令、原始输出、修改前后请求样例和一段消息数量对比。若只读过实验，写“理解协议”；实际跑过后写“复现重复回传”；亲自补缺陷测试后才写“改进幂等边界”。

5.16 面试表达：流式与续跑不是两个名词

30 秒概念回答：SSE 让一次 HTTP 响应陆续返回事件。LR-Agent 把模型和工具事件适配为稳定业务协议。外部工具未在后端执行时发 tool_pending；下一请求带历史与同 ID 的结果，后端补 ToolMessage 并播种已完成调用集合。当前这是一种请求重建续跑，没有图 checkpointer。

2 分钟项目回答框架：先讲标注任务里为什么要交接给外部执行方，再讲 API 校验、官方 Agent 循环、事件适配、pending/result 配对和用户确认状态。用实验二解释消息去重，再说明执行抑制与取消清理分别在哪。最后主动交代局限：进程内取消、输入历史依赖，以及没有验证全局 exactly-once。

追问	回答要点	可出示证据
为什么不持有暂停的请求？	跨请求用显式数据重建	pending→resume 时序图
为什么不直接图持久恢复？	当前选择简单协议、调用方带历史	factory 未传 checkpointer
服务重启后怎样继续?	依赖调用方再次提交必要数据	源码职责，尚非故障演练
你自己做了什么?	分清阅读、复现、亲自改造	本人脚本输出或提交

本章事实边界

源码已确认：路由、帧编码、消息重建、完成 ID 播种和协作取消。本次已执行：三个纯离线实验。已有真实验收另见项目记录；本章未调用真实模型、未写标注文件，也未验证代理缓冲和分布式恢复。

5.17 源码图解：从函数签名读到返回值

先看图中的真实路径与原始行号，再打开源文件跟读。签名说明输入条件，函数体说明处理，return说明交给下一层的数据。图为源码阅读辅助，不替代完整上下文。

真实源码阅读图 / 路由依赖与健康检查

LR-Agent/vendor/local-agent/local_main.py

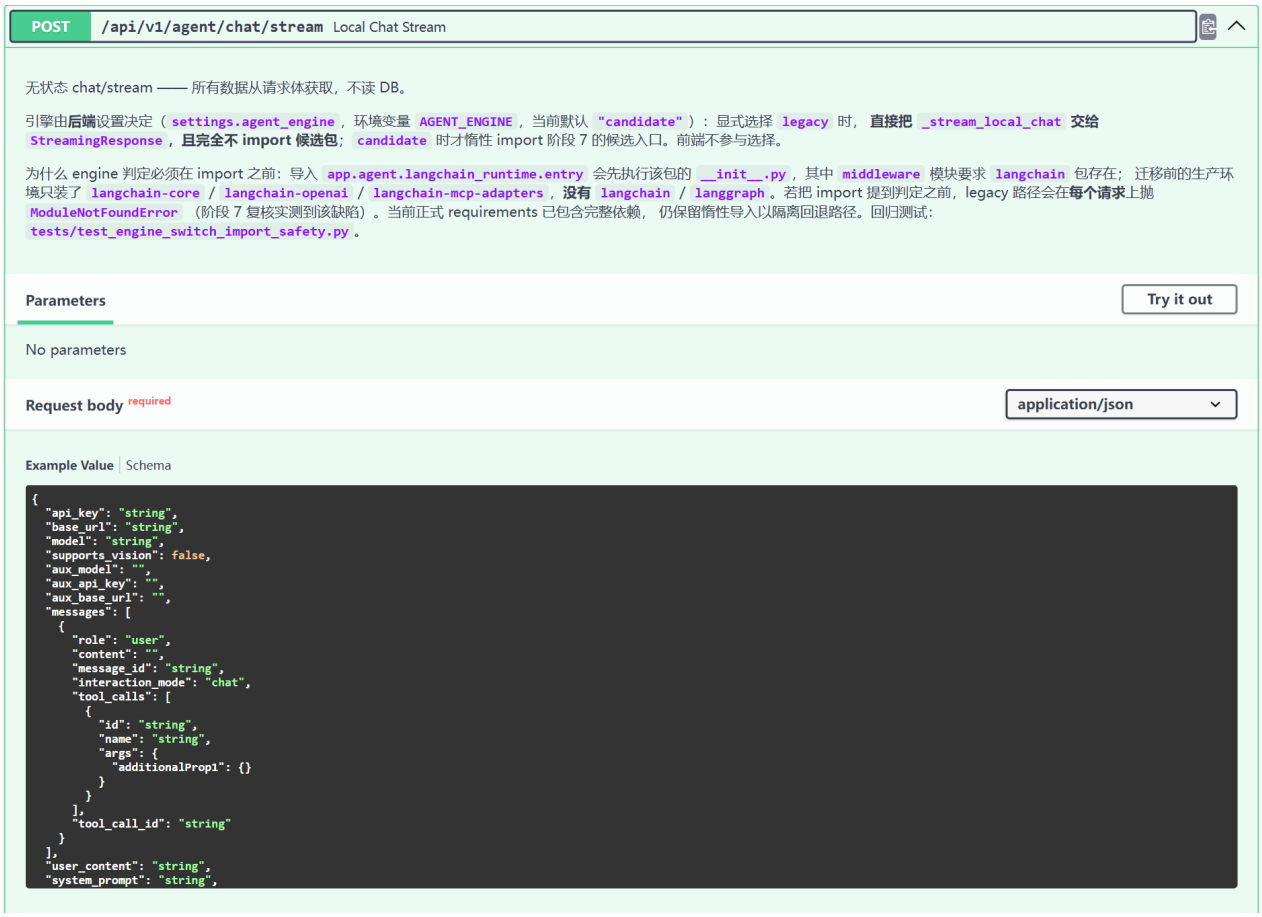
```
57      app.include_router(
58          api_router,
59          prefix=settings.api_v1_prefix,
60          dependencies=[Depends(require_local_token)],
61      )
62
63      @app.get("/health", tags=["health"])
64      async def health_check() -> dict[str, str]:
65          return {"status": "ok"}
66
67      return app
```

真实源码阅读图：LR-Agent/vendor/local-agent/local_main.py::create_app。按当前文件原始行号截取并排版生成。

先看什么	怎样读	自己记录什么
输入	区分实参、配置与上下文	哪些字段可缺失
处理	找条件、调用和错误分支	哪一步改变消息或执行
输出	确认返回值与接收者	怎样用实验检查

5.18 真实接口图：把后端符号与请求对上

本轮从当前源码启动了仅监听本机的文档实例，截图来自真实Swagger页面。此处只读查看，没有提交模型请求、推理任务或个人凭据。动态端口不属于协议语义。



本轮真实截图：chat/stream接口及自动生成示例；string均为文档占位值。

图中信息	对应后端问题	继续核对
接口路径	由哪个函数接收请求	查路由装配与前缀
请求结构	模型能读到哪些数据	查Schema与消息转换
状态与结果	什么是完成、失败或挂起	查SSE与续跑合同

源码与证据索引

LR-Agent/vendor/local-agent/app/api/v1/agent.py::local_chat_stream/cancel_chat

LR-Agent/vendor/local-agent/app/core/deps.py::SettingsDep

LR-Agent/vendor/local-agent/app/schemas/agent.py::LocalChatStreamRequest/ClientToolResult/StreamEventPayload

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/entry.py::sse_frame/stream_candidate_engine_sse

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/resume.py::prepare_resume_turn/prepare_resume_messages

LR-Agent/vendor/local-agent/app/agent/context_service.py::enrich_resume_tool_result

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/runtime.py::build_langchain_agent

LR-Agent/vendor/local-agent/app/agent/langchain_runtime/sse_adapter.py::stream_agent_events

阅读时先定位符号，再看完整输入、处理与返回。源码存在、离线实验通过和已有真实验收属于不同证据；未验证的模型质量、效率与并发结论仍需单独实验。